Daniel Review

TailEnhancedFinDiff — Full Research Review

> **Date:** 2 April 2026
> **Repo:** `repos/TailEnhancedFinDiff` (Renyi Yang, MIT License)
> **Reference paper:** https://arxiv.org/abs/2407.18609 (DLPM)

---

1. What This Research Does

This repo implements a **factor-space diffusion model** for generating synthetic stock return scenarios. Instead of running a diffusion model directly on stock returns (high-dimensional, hundreds of stocks), it:

1. Decomposes stock returns into **7 factor returns** via cross-sectional OLS
2. Trains a **denoising diffusion model** on the 7-dimensional factor return time series
3. Samples new factor vectors from the trained model
4. **Reconstructs** stock returns from synthetic factors via $R = F\beta + \varepsilon$

The key innovation over a plain DDPM is using **Lévy α-stable noise** (DLPM) instead of Gaussian noise in the forward/reverse diffusion, which better captures the heavy tails empirically observed in financial returns.

---

2. The Factor Model (`factor_model.py`)

2.1 Factor Construction

Regression factors (`build_regression_factors`): For each date t, runs cross-sectional OLS:

$$R_t = B_{t-1} f_t + \varepsilon_t$$

where B_{t-1} is an $(N \times 7)$ matrix of **lagged** stock characteristics (ones-column for market intercept, plus growth, momentum, quality, size, value, volatility). Solves for $f_t \in \mathbb{R}^7$ via `np.linalg.lstsq`. This produces a time series of 7-dimensional factor returns.
An `alpha` column of constant 1.0 is prepended to F — this acts as the intercept in the second-stage time-series regression.

Portfolio-sort factors (`build_portsort_factors`): Alternative construction. Sorts stocks into 5 quantile portfolios by each characteristic, computes long-short spread (Q5 − Q1). Produces cleaner zero-investment factor series but is not used in the main pipeline.

2.2 Beta Fitting (`fit_beta`)

Per-stock time-series OLS with **next-date alignment**: factor returns at date t predict stock returns at $t+1$.

$$r_s = F\beta_s + \varepsilon_s \quad \text{for each stock } s$$

Handles NaN via per-stock masking. Stores:

- `beta`: $(K \times S)$ coefficient matrix

- `res_std`: per-stock residual standard deviation
- `residuals`: $(T \times S)$ residual matrix
- `res_df`: degrees of freedom from fitting a **t-distribution** to pooled standardised residuals (uses `scipy.stats.t.fit` with `floc=0`)

2.3 Return Reconstruction (`reconstruct_returns`)

$$r_i^{(s)} = f^{(s)\top}\beta_i + \sigma_i \cdot t_\nu \cdot \sqrt{(\nu-2)/\nu}$$

The $\sqrt{(\nu-2)/\nu}$ scaling ensures the t-noise has unit variance, matching the fitted `res_std`. Residuals are drawn i.i.d. across stocks — no cross-sectional correlation in the idiosyncratic component.

---

3. The Diffusion Model

3.1 Architecture: FactorDenoiser (`factor_diffusion_train.py`)

A Transformer-based denoiser that treats the 7 factors as a **sequence of tokens**:

```
x_t ∈ R^7  →  unsqueeze to R^(7×1)
           →  Linear(1, 64) per factor  →  h ∈ R^(7×64)
           →  + learnable factor embeddings (7×64 parameter)
           →  BasicTransformerBlock(dim=64, 4 heads, AdaNorm)
           →  Linear(64, 1) per factor  →  ε^ ∈ R^7
```

Timestep conditioning: sinusoidal encoding → 2-layer MLP → 128-dim condition vector, injected via AdaNorm (adaptive normalisation, same mechanism as class-conditional image diffusion). Only **1 transformer block** (configurable via `num_blocks`). Very lightweight model for a 7-dim problem.

There is also a commented-out **MLP variant** (`FactorDenoiserMLP`) and a full **FactorizedDDPM** architecture in `sample_code.py` that operates on the full stock×time tensor (81 stocks × 22 days) with dilated temporal convolutions + cross-asset attention. This more complex architecture is NOT used — it appears to be exploratory/future work.

3.2 Noise Schedule (`factor_diffusion_levy.py`)

Cosine schedule generalised for stability index α:

$$\gamma_t = \alpha_t^{1/\alpha}, \quad \bar{\gamma}_t = \prod_{s=0}^{t} \gamma_s, \quad \sigma_t = (1-\gamma_t^\alpha)^{1/\alpha}, \quad \bar{\sigma}_t = (1-\bar{\gamma}_t^\alpha)^{1/\alpha}$$

When $\alpha = 2$: reduces exactly to standard DDPM cosine schedule.

3.3 Forward Process (Training)

DDPM mode:

$$x_t = \bar{\gamma}_t x_0 + \bar{\sigma}_t \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

Loss: $\|\text{model}(x_t, t) - \varepsilon\|^2$

DLPM mode:

$$x_t = \bar{\gamma}_t x_0 + \sqrt{a_t \cdot \bar{\sigma}_t^2} \cdot z, \quad a_t \sim S(\alpha/2, 1), \quad z \sim \mathcal{N}(0, I)$$

The target becomes $\varepsilon_t = (x_t - \bar{\gamma}_t x_0)/\bar{\sigma}_t = \sqrt{a_t} \cdot z$, which now has **random variance** a_t.

Median-of-means loss: Because Lévy samples can be extreme, the loss is estimated via:

- MC_OUTER=3 independent a_t draws (Lévy variance multipliers)
- MC_INNER=1 Gaussian draw per a_t
- MSE per sample → mean over inner → **median** over outer
- Median is robust to the occasional extreme a_t that would blow up a mean

3.4 Reverse Process (Sampling, `factor_diffusion_sample.py`)

The DLPM reverse process is more complex than DDPM:

1. **Pre-sample** an entire Lévy variance path $A_0, A_1, \ldots, A_{T-1}$ (one per timestep per sample)
2. **Build conditional variance chain:** $\Sigma_0 = \sigma_0^2 A_0$, then $\Sigma_t = \sigma_t^2 A_t + \gamma_t^2 \Sigma_{t-1}$
3. **Initialise** from heavy-tailed noise: $x_T = \bar{\sigma}_{T-1} \cdot \sqrt{a_{init}} \cdot z$ where $a_{init} \sim S(\alpha/2, 1)$
4. **Denoise** from $t = T - 1$ to 1:

$$\Gamma_t = \text{clamp}\left(1 - \frac{\gamma_t^2 \Sigma_{t-1}}{\Sigma_t + 10^{-8}}, [0, 1]\right)$$

$$\mu_t = \frac{x_t - \bar{\sigma}_t \Gamma_t \hat{\varepsilon}(x_t, t)}{\gamma_t}$$

$$V_t = \text{clamp}(\Gamma_t \cdot \Sigma_{t-1}, \min = 0)$$

$$x_{t-1} = \mu_t + \sqrt{V_t} \cdot \mathcal{N}(0, I) \quad (t > 1); \quad x_0 = \mu_1 \quad (t = 1)$$

3.5 Conditional Generation (`generate_conditional`)

Classifier-free guidance for generating scenarios where a specific factor exceeds a threshold:

1. At each timestep, estimate $\hat{x}_0$ from current x_t and predicted noise
2. Compute soft constraint: $\mathcal{L} = \|\text{ReLU}(\text{threshold} - \hat{x}_{0,\text{factor}})\|^2$
3. Backprop through x_t (not through the model — noise prediction is detached)
4. Shift: $x_t \leftarrow x_t - \lambda \cdot \nabla_{x_t} \mathcal{L} / \|\nabla_{x_t} \mathcal{L}\|$
   Important: This function uses a `scheduler` object (from `diffusers` library DDPM scheduler) — it is NOT compatible with the DLPM noise schedule. Conditional generation only works in DDPM mode.

3.6 Lévy Sampling (`factor_diffusion_levy.py`)

Uses the **Chambers-Mallows-Stuck (CMS) algorithm** for sampling $S(\alpha/2, \beta = 1)$:

- Uniform angle $\Theta \sim U(-\pi/2, \pi/2)$, exponential weight $W \sim \text{Exp}(1)$
- Trigonometric transformation to produce positive right-skewed stable samples
- Oversamples 2× and takes first n finite samples
- Clamps to $[0, 2000]$ for numerical stability

The SαS (symmetric alpha-stable) sampling is via the **Gaussian-Lévy scale mixture**: $\varepsilon = \sqrt{a} \cdot z$ where $z \sim \mathcal{N}(0, I)$ and $a \sim S(\alpha/2, 1)$.

4. Training Configuration

Parameter	Value	Notes
Epochs	200	
Batch size	64	
Learning rate	1e-4	Cosine annealing to 0
Optimizer	AdamW	weight_decay=1e-4
Timesteps	100	
Lévy α	1.9	Close to Gaussian (2.0) but with heavier tails
MC outer	3	Lévy variance draws
MC inner	1	Gaussian draws per variance
Gradient clipping	norm ≤ 1.0	
Data normalisation	StandardScaler	Per-factor z-score
Num. generated	4096	Sliced to 2048 for evaluation

Data: factor returns CSV (extracted from `build_regression_factors`), column subset = [market, growth, momentum, quality, size, value, volatility]. Rows with any NaN are dropped.

5. Experiments

Experiment 1: Cross-Sectional Distribution Quality (`experiments.md`)

Setup: StockNet data. Generate 2048 scenarios from FactorDM and Stationary Bootstrap. Compare generated [2048, 81] matrices against training data [1257, 81].
Convergence test: Generate 4096 samples, slice to [32..4096]. Track absolute mean deviation of the highest-variance stock. Both methods stabilise around 2048 samples.
Results (from `experiments.md`):

Metric	FactorDM	Bootstrap	Winner
W1	0.0011	**0.0003**	Bootstrap
Cov error	0.1943	**0.0785**	Bootstrap
ES 5%	-0.098±0.05	**-0.026±0.04**	Bootstrap
Mean	-0.495±3.00	**0.391±1.56**	Bootstrap
Std	-0.101±0.03	**-0.008±0.03**	Bootstrap
Skew	-1.357±3.26	2.275±17.37	Mixed
Kurt	-0.363±0.28	**-0.042±0.23**	Bootstrap

Bootstrap wins on every distribution metric. This is expected — bootstrap literally resamples real data.

Diversity/novelty: FactorDM generates genuinely **novel** scenarios (nearest-neighbour distance to training ≈ 0.065). Bootstrap NN distance = 0 (exact resamples). This is the core value proposition: generate plausible scenarios that haven't been observed.

Portfolio risk (downstream, equal-weight):

Metric	Historical	FactorDM	Bootstrap
VaR 5%	-0.0120	-0.0114	-0.0109
ES 5%	-0.0168	-0.0164	-0.0158
Vol	0.0072	0.0066	0.0070
MaxLoss	-0.0385	-0.0350	-0.0367

Both methods produce well-calibrated risk estimates.

Experiment 2: Temporal Dynamics (`experiment2.md`)

Setup: StockNet windowed into [M, 81, 22] (22-day paths). Compare Stationary Bootstrap vs MacroVAE (external baseline).
Results:

Method	W1	Cov	LinUnpred	VolClust	CoarseFine
Bootstrap	0.0000	0.0160	-0.007	+0.001	-0.212
MacroVAE	0.0021	0.6190	-0.067	-0.072	-0.719

Bootstrap near-perfect. MacroVAE struggles badly with covariance and temporal structure. **The DLPM/FactorDM is NOT tested here** — the model currently generates single cross-sections, not temporal paths.

Stock-Level Evaluation (`stock_evaluation.ipynb`)

Most complete evaluation. Uses train/test split:

1. **Per-stock moment scatter plots** — (Mean, Std, Skew, Kurt) of generated vs actual (train and OOS), with Pearson R^2
2. **Per-stock KS tests** — 2-sample KS between generated and actual per stock. Reports fraction with $p > 0.05$
3. **Cross-sectional dispersion decomposition** — separates systematic ($F\beta$) and idiosyncratic noise, compares CS-std distributions for Generated vs Train vs OOS
4. **Wasserstein-1 distances** of CS-std distributions

Factor-Level Evaluation (`factor_evaluation.ipynb`)

Compares generated factor distributions against OOS:

1. Mahalanobis Q-Q plots (multivariate normality check)
2. Per-factor moment comparison with MAE
3. KDE distribution overlays per factor
4. Per-factor KS tests
5. Correlation matrix heatmaps + Frobenius distance
6. PCA projection (2D)

Baselines: Gaussian MVN sampling and Resample (random rows from training).

Factor Model Evaluation (`factor_model_eval.ipynb`)

Exploratory notebook that:

- Builds regression and portfolio-sort factors from scratch

- Visualises residual distributions (shows heavy tails vs Gaussian fit)
- Tests factor sampler (MVN) and diffusion-generated samples
- Plots Mahalanobis Q-Q for regression factors, MVN, and DDPM — visually checks for multivariate normality departures
- Computes kurtosis breakdown: factor part, residual part, total, sampled

Convergence Experiment (`convergence_experiment.ipynb`)

Generates 4096 samples, slices to increasing sizes, tracks mean convergence of highest-variance stock. Used to justify the 2048 sample size.

Stationary Bootstrap (`stationary_bootstrap.ipynb`)

Implements two versions:

- **Cross-sectional:** generates one Markov chain bootstrap path, then samples N rows from it
- **Temporal:** generates M independent 22-day bootstrap paths [M, N, D]

---

6. Bugs, Issues, and Concerns

6.1 CRITICAL: `reconstruct_return` in notebook uses Gaussian residuals, not t-distributed

`factor_model_eval.ipynb` cell 18 defines a local function:

```
def reconstruct_return(fs, beta, res_std):
    N = fs.shape[0]
    return (fs @ beta) + np.random.normal(0, 1, size=(N, len(res_std))) * res_std[None, :]
```

This uses **Gaussian** residuals. But the module-level `reconstruct_returns` in `factor_model.py` uses **t-distributed** residuals with the fitted degrees of freedom ($\nu \approx 3$). The saved sample files `PSM2048.npy`, `PSMDiff2048.npy`, `PSMResample2048.npy` (cells 23-24, 26) are all generated using the Gaussian version.

Impact: Any evaluation comparing these baselines will understate their tail heaviness, making the diffusion model appear relatively better at capturing tails than it actually is. This is a significant experimental confound.

6.2 Stationary Bootstrap implementation is wrong for cross-sectional experiment

In `stationary_bootstrap.ipynb`, cells 4-5:

```
# Cell 4: generates ONE Markov chain path of length T
for t in range(data.shape[0]):
    out[t] = data[idx]
    if rng.random() < p: idx = rng.integers(0, data.shape[0])
    else: idx = (idx + 1) % data.shape[0]
# Cell 5: i.i.d. sample N rows from this single path
final = sample(out, N)
```

This generates **one** stationary bootstrap path and then randomly samples 2048 rows from it. The `sample()` function (cell 3) does pure i.i.d. row selection — destroying any temporal structure. A

proper multivariate stationary bootstrap would generate each of the 2048 scenarios as an independent draw, or at minimum would preserve the block structure.

The temporal version (`sample_temporal` in cell 7) is correct — each scenario gets its own Markov chain. But the cross-sectional version used for experiment 1 is methodologically flawed.

Impact: Since only cross-sectional (i.i.d.) metrics are evaluated, the impact is somewhat mitigated — the final samples still come from the empirical distribution. But the "stationary bootstrap" label is misleading; it's really "resample from a permuted copy of the data."

6.3 No train/test split in experiment 1

`experiments.md` (experiment 1) evaluates generated samples against **training data**. The factor model is fit on training data, the diffusion model is trained on factor returns from training data, and generated samples are compared back to training data. This measures in-sample replication fidelity, not out-of-sample generalisation.

`stock_evaluation.ipynb` and `factor_evaluation.ipynb` correctly use separate test sets — but the headline results reported in `experiments.md` are in-sample.

6.4 Configuration mismatch between docs and code

`experiments.md` says: "For Factor DM, we follow the default training/sampling setting: batch size 32, learning rate 1e-4, **600 epochs**, cosine noise scheduler, **200 sampling steps**."
The code defaults to: `BATCH_SIZE = 64`, `EPOCHS = 200`, `NUM_TIMESTEPS = 100`, `MODE = "DLPM"`.
The experiments doc describes DDPM with different hyperparameters. It is unclear which configuration produced the reported results. This makes the experiments non-reproducible.

6.5 No cross-stock correlation in residuals

`reconstruct_returns` draws residuals independently per stock:

```
noise = np.random.standard_t(model.res_df, size=(N, S))
```

In reality, factor model residuals have cross-sectional correlation — factor models don't perfectly absorb all common variation. This means generated covariance comes only from the $F\beta$ term, and the idiosyncratic component adds only diagonal variance. This likely explains the Cov error being 0.1943 vs 0.0785 for bootstrap.

6.6 `CHECKPOINT` path has fragile zero-padding

```
CHECKPOINT = PREFIX + f"/checkpoints/factor_{MODE}_ep0{EPOCHS}.pt"
```

This produces `ep0200.pt` which accidentally matches the training output `ep{200:04d}.pt` = `ep0200.pt`. But for ≥1000 epochs, the path would become `ep01000.pt` vs the saved `ep1000.pt`, breaking loading.

6.7 The "alpha" column semantics are confusing and error-prone

- `build_regression_factors` returns F with columns `["market"] + FEATURES` (7 cols), then **inserts** `alpha = 1.0` at position 0 → 8 columns
- `load_data` for diffusion training reads only `FACTOR_NAMES` = 7 cols (**excluding** `alpha`) → diffusion model is 7-dimensional
- `stock_evaluation.ipynb` re-prepends ones: `np.column_stack([np.ones(...), sampled_factors])` → 8 cols for reconstruction

This works but is fragile. If anyone changes column order or forgets the re-prepend, betas will be silently misaligned. The name "alpha" for a constant-1 intercept column is also misleading (sounds like Jensen's alpha).

6.8 Conditional generation only works for DDPM, not DLPM

`generate_conditional` uses `scheduler.alphas_cumprod` and `scheduler.step()` from the `diffusers` DDPM scheduler. These are DDPM-specific. Since the default mode is DLPM, this function is effectively dead code in the current configuration.

6.9 No random seed management

No `torch.manual_seed()` or `np.random.seed()` in the training script. No seed in the sampling script. Results are not reproducible across runs. Only `stock_evaluation.ipynb` and `convergence_experiment.ipynb` use `np.random.default_rng(42)` for some operations.

6.10 Data leakage in factor_model_eval.ipynb

Cell 27 generates returns using `FF.values @ beta_r` with factor returns and betas both fitted on the same data, saving the result as `PSMExactlys3913.npy`. This is used elsewhere as if it were a generative sample — but it's a reconstruction from the training data itself with only the residuals re-drawn.

6.11 FactorizedDDPM (`sample_code.py`) is disconnected

A complete stock×time architecture (81 stocks × 22 days) with dilated temporal convolutions and cross-asset attention exists but is never called. The `__main__` block runs it and it works. This appears to be the next planned step (temporal multi-day generation) but was never integrated.

---

7. Evaluation Metrics —

Cross-sectional metrics (`metrics/statistic.py`)

Metric	Computation	Interpretation
W1	Per-asset 1D Wasserstein-1 distance, averaged over assets	Marginal distribution closeness. Lower = better
Cov	$\|\hat{\Sigma}_X - \hat{\Sigma}_Y\|_F / \|\hat{\Sigma}_Y\|_F$	Correlation structure error. Very sensitive to # samples
ES	Mean of returns below 5th percentile, per asset, then mean±std	Left tail quality. Critical for risk
Mean	Per-asset sample mean, then mean±std across assets	Central tendency
Std	Per-asset sample std, then mean±std across assets	Volatility matching
Skew/Kurt	Per-asset scipy.stats.skew/kurtosis, then mean±std	Shape of distribution

Temporal metrics (`metrics/statistic_temporal.py`)

Metric	Computation
LinUnpred	Mean ACF of returns at lags 1-10 (efficient markets → ≈0)

Metric	Computation
VolClust	Mean ACF of |returns| at lags 1-10 (should be >0, captures volatility persistence)
CoarseFine	Cross-correlation between $\|\sum r\|$ (coarse vol) and $\sum \|r\|$ (fine vol) at lags
Leverage	$(E[r_t \cdot r_{t+k}^2] - E[r_t]E[r_t^2])/E[r_t^2]^2$, measures return-volatility asymmetry

Portfolio risk metrics (`metrics/risk.py`)

VaR (5th percentile), ES (mean below VaR), Vol (std), MaxLoss (min return) — computed on portfolio return $r_p = Xw$.

---

8. Architecture of `FactorizedDDPM` (`sample_code.py`)

This is a complete but unused architecture for **temporal multi-day, multi-stock** generation. It operates on tensors of shape [B, 81, 22]:

- **Input projection:** Return → 64-dim hidden per stock per day
- **Stock embeddings:** Learnable per-stock (81 embeddings)
- **Temporal positional embeddings:** Learnable per-day (22 embeddings)
- **FactorizedBlock** (×3 layers):
  - Pathway A: Dilated temporal convolution (dilations 1,2,4) along time axis
  - Pathway B: Multi-head cross-asset attention along stock axis
  - Combine via AdaLayerNorm with timestep conditioning
- **Output:** Predicted noise [B, 81, 22]

Total parameters: printable via `__main__` block. This architecture addresses the main limitation of the current system (no temporal structure) but was never connected to training data or evaluation.

---

9. Summary

What's really good!!

1. **Factor-space diffusion is dimensionally efficient** — 7-dim problem instead of 81-491
2. **Lévy noise is theoretically motivated** for financial tails - VERY VERY NICE!!!!!!
3. **t-distributed residuals** in reconstruction (module version) capture per-stock fat tails
4. **Rich evaluation suite** — 15+ metrics across 3 dimensions (distributional, temporal, portfolio)
5. **Well-structured notebooks** for factor-level and stock-level evaluation
6. **Conditional generation** via gradient guidance is powerful for stress testing
7. **The FactorizedDDPM** in `sample_code.py` is a promising next step

What needs some thinking

1. **Gaussian vs t-distributed residual inconsistency** (§7.1) — invalidates baseline comparisons
2. **Wrong bootstrap implementation** for cross-sectional experiment (§7.2)
3. **Experiment 1 is in-sample evaluation** (§7.3) — needs OOS results
4. **Configuration mismatch** between docs and code (§7.4) — experiments aren't reproducible as described
5. **No residual cross-correlation** (§7.6) — covariance structure is fundamentally limited

6. **Conditional generation only works for DDPM** (§7.10) — the headline DLPM mode can't do it

Key research questions

1. How much tail improvement comes from DLPM noise vs t-distributed residuals? Need ablation: DLPM+Gaussian vs DDPM+t
2. Can residual cross-correlation be modelled (correlated t-draws, copula, or diffuse in residual space)?
3. Can the FactorizedDDPM architecture be trained for temporal (multi-day) scenario generation?
4. How does performance change with more factors (40 vs 7)?
5. What is the right α for DLPM? Only α=1.9 is tested — is this optimal or was it tuned?

---

10. Conditional Scenario Generation — What Needs To Be Done

11.1 Current state of `generate_conditional()`

The function in `factor_diffusion_sample.py` implements **classifier-free gradient guidance** during the reverse diffusion process. It works like this:

1. Start from pure Gaussian noise `x_T`
2. At each timestep, predict noise → estimate clean sample `x0_hat`
3. Compute a penalty: `relu(threshold_norm - x0_hat[:, factor_idx])^2` — i.e., penalise samples where the conditioned factor is *above* the threshold (for a "down" scenario)
4. Compute gradient of that penalty w.r.t. `x`
5. Take a normal DDPM reverse step, then shift `x` in the direction that reduces the penalty

Critical problem: This function uses `scheduler.alphas_cumprod[t_val]` which is a DDPM-specific attribute. The default training mode is DLPM. So you **cannot use conditional generation with the default trained model**. The DLPM reverse process in `generate()` uses a completely different variance chain (Σ_t from pre-sampled Lévy subordinators A_t), not `alphas_cumprod`.

11.2 What needs to be built

A. Port conditional generation to DLPM

The gradient guidance mechanism needs to be adapted to the DLPM reverse process:

```
def generate_conditional_dlpm(model, gammas, bargammas, sigmas, barsigmas,
                              levy_alpha, scaler,
                              factor, threshold, guidance_scale=200.0):
    """
    DLPM reverse process with gradient guidance for conditional generation.
    """
    factor_idx = FACTOR_NAMES.index(factor)
    # Transform threshold to standardised space
    dummy = np.zeros((1, FACTOR_DIM))
    dummy[0, factor_idx] = threshold
    threshold_norm = float(scaler.transform(dummy)[0, factor_idx])
    T = len(gammas)

    # ... same setup as generate() ...

    for t in range(T - 1, 0, -1):
```

```
        x = x.detach().requires_grad_(True)
        t_b = torch.full((n,), t, dtype=torch.long, device=DEVICE)

        with torch.no_grad():
            eps_pred = model(x, t_b)

        # Estimate x0 using DLPM posterior mean (not DDPM formula!)
        Gamma_t = 1 - (gammas[t]**2 * Sigmas[t-1]) / (Sigmas[t] + 1e-8)
        Gamma_t = Gamma_t.clamp(0.0, 1.0)
        x0_hat = (x - barsigmas[t] * Gamma_t * eps_pred) / gammas[t]

        # Conditional penalty (same idea)
        cond_loss = torch.relu(threshold_norm - x0_hat[:, factor_idx]).pow(2).mean()
        grad = torch.autograd.grad(cond_loss, x)[0]

        with torch.no_grad():
            # Normal DLPM reverse step
            mean = (x - barsigmas[t] * Gamma_t * eps_pred) / gammas[t]
            var = (Gamma_t * Sigmas[t-1]).clamp(min=0.0)
            if t > 1:
                x = mean + var.sqrt() * torch.randn_like(x)
            else:
                x = mean

            # Apply guidance
            grad_norm = grad.norm(dim=-1, keepdim=True).clamp(min=1e-8)
            x = x - guidance_scale * grad / grad_norm
```

Key difference: The x0 estimate formula is different for DLPM vs DDPM. In DDPM it's `(x - sqrt(1-αbar)*eps) / sqrt(αbar)`. In DLPM it's `(x - barsigmas[t]*Gamma_t*eps) / gammas[t]`. Using the wrong formula will produce garbage x0 estimates and the guidance will not work.

B. Define the conditioning interface

Currently the function takes a single `(factor, threshold)` pair meaning "this factor should be *below* this threshold". For a quant team, you want:

1. **Single-factor stress**: "momentum return = -3σ" → condition momentum factor to be below -3σ
2. **Multi-factor stress**: "momentum down AND volatility up" → sum penalties for multiple factors
3. **Joint scenario**: "market crash" = simultaneous conditioning on market < -2σ, volatility > +2σ, quality > 0
4. **Magnitude control**: Instead of a hard threshold, allow a soft target: "momentum ≈ -2σ" using a Gaussian penalty `(x0_hat[:, idx] - target)^2` instead of `relu(threshold - x0_hat[:, idx])^2`

Concrete extension:

```
def generate_conditional_multi(model, ..., conditions, guidance_scale=200.0):
    """
    conditions: list of dicts, each with:
        - factor: str (e.g. "momentum")
        - target: float (raw return space, e.g. -0.05 for -5%)
        - direction: "below" | "above" | "at"
        - weight: float (relative importance, default 1.0)
    """

    for cond in conditions:
        idx = FACTOR_NAMES.index(cond["factor"])
        target_norm = scaler.transform(...)
        if cond["direction"] == "below":
```

```
            loss += cond["weight"] * relu(target_norm - x0[:, idx]).pow(2).mean()
        elif cond["direction"] == "above":
            loss += cond["weight"] * relu(x0[:, idx] - target_norm).pow(2).mean()
        elif cond["direction"] == "at":
            loss += cond["weight"] * (x0[:, idx] - target_norm).pow(2).mean()
```

C. End-to-end pipeline: from factors to stock returns

Right now `generate_conditional()` returns factor returns only. To get stock-level impact you need to chain:

1. **Generate conditional factor samples** → shape `(N, 7)` daily factor returns
2. **Reconstruct stock returns** using `factor_model.py`:
   - Load saved betas: `β_i` for each stock (shape `(num_stocks, 7)`)
   - Sample residuals: `ε_i ~ t(ν_i, 0, σ_i)` (per-stock fitted t-distribution)
   - Compute: `r_i = β_i @ f + ε_i` for each sample
3. **Result**: `(N, num_stocks)` stock returns conditioned on the factor scenario

Missing piece: The residual parameters (ν, σ per stock) and betas are saved when you run `factor_model.py` but there's no clean function to load them and do reconstruction from arbitrary factor samples. You need a `reconstruct_from_factors(factor_samples, beta_path, residual_params_path)` utility.

D. Guidance scale tuning

The `guidance_scale=200.0` is hardcoded. Too low → condition is ignored. Too high → samples collapse to a single point (zero diversity). You need to:

1. Sweep guidance_scale in [50, 100, 200, 500, 1000]
2. For each: measure (a) how well the condition is satisfied (% of samples meeting threshold), (b) sample diversity (cross-sectional variance of other factors)
3. Pick the scale that satisfies the condition for >90% of samples while preserving factor diversity

11.3 Minimal test plan for conditional generation

Step	What	Validates
1	Train DDPM model (set MODE="DDPM")	Baseline that works with existing `generate_conditional()`
2	Run `generate_conditional(model, scheduler, scaler, "momentum", -0.03)`	Generates 4096 samples where momentum < -3%
3	Plot histogram of momentum factor in output	Visual check: mass should be concentrated below -3%
4	Plot histogram of other 6 factors	They should NOT all collapse — must retain realistic spread
5	Compare unconditional vs conditional: KS test on each factor	Momentum should shift, others should be similar
6	Reconstruct stock returns from conditional factors	Stock returns should show momentum-crash characteristics
7	Repeat with DLPM model using new `generate_conditional_dlpm()`	Validates the DLPM port works
8	Compare DDPM conditional vs DLPM conditional tail statistics	DLPM should produce fatter tails in the non-conditioned factors

12. Visualisation for the Equity Quant Team

12.1 Core visualisations

A. Conditional vs unconditional factor distributions (ridgeline / violin plots)

For each of the 7 factors, show two distributions side by side:

- **Blue**: unconditional samples (what the model generates normally)
- **Red**: conditional samples (e.g., "momentum < -3%")

This immediately shows: (a) the conditioned factor shifted as expected, (b) which other factors are pulled along (cross-factor dependence), and (c) the shape of the tails.

```
Factor         Unconditional          Conditional (mom < -3%)

_____________________________________________________________

market         ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒       ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒

growth         ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒       ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒

momentum       ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒       ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒   ← shifted left

quality        ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒       ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒

size           ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒       ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒

value          ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒       ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒

volatility     ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒       ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒   ← moved?
```

The key insight a quant team wants: **does conditioning on one factor drag other factors in the right direction?** If the model has learned cross-factor structure, a momentum crash should also show elevated volatility and possibly depressed market — matching historical behaviour.

B. Factor correlation heatmaps: unconditional vs conditional

Two heatmaps side by side:

- Left: empirical correlation of unconditional generated samples
- Right: empirical correlation of conditional samples

Show how the correlation structure changes under stress. In a real momentum crash, cross-factor correlations typically increase (everything becomes more correlated). Does the diffusion model capture this? If not, it's producing unrealistic scenarios.

C. Stock-level return distribution under stress

After reconstructing stock returns from conditional factors:

1. **Portfolio return histogram**: Show Returns/P&L distribution of a specific portfolio (e.g., s&p500, long momentum, equal weighted factor portfolio) under the stress scenario vs under unconditional
2. **VaR / ES shift table**:

Metric	Unconditional	Momentum crash	Difference
VaR 1%	-2.3%	-4.1%	-1.8%
ES 1%	-3.5%	-6.2%	-2.7%
Mean	+0.02%	-1.8%	-1.82%

3. **Stock-level scatter**: x-axis = stock's momentum beta, y-axis = stock's conditional expected return. Should show a clear negative slope (high momentum-beta stocks lose more in the scenario).

D. Scenario comparison dashboard - I can build this

A single figure with 4 panels comparing different stress scenarios:

Panel	Scenario	Condition
Top-left	Baseline (unconditional)	None
Top-right	Momentum crash	momentum < -3%
Bottom-left	Vol spike	volatility > +3%
Bottom-right	Combined	momentum < -3% AND volatility > +3%

Each panel shows the portfolio return distribution. This lets the quant team immediately see whether the combined scenario is worse than the sum of individual scenarios (non-linear interaction = the model has learned something useful).

E. Tail comparison: diffusion vs historical

Overlay the conditional samples with actual historical days that match the condition:

- Filter historical data for days where momentum factor was below -3%
- Plot both on the same axes
- If the diffusion model is good, its conditional distribution should **envelope** the historical one (more diverse because it generates plausible scenarios that haven't happened yet) while having similar location and scale

12.2 Presentation narrative for quant team

The story to tell:

1. "We trained a generative model on historical factor returns that captures the full joint distribution including fat tails and cross-factor dependence"
2. "This is what diffusion models are and this is the enhancement we made to our one (levy distribution)"
3. "We can condition the model to generate scenarios: tell it 'momentum crashes 3%' and it generates 4096 plausible daily factor returns consistent with that event"
4. "The non-conditioned factors move realistically — volatility goes up, correlations increase — because the model learned the joint structure"
5. "We reconstruct stock-level returns from these stressed factors and measure portfolio impact"
6. "This is more powerful than historical stress tests (limited to events that actually happened) and more realistic than parametric stress tests (which assume Gaussian or independent factors)"

---

13. Evaluating the Added Power of the Diffusion Model

13.1 The key question

> "Is a diffusion model better than simpler alternatives for generating realistic conditional scenarios?"
> To answer this rigorously, test **baselines** to compare against and **metrics** that measure what matters for a quant team.

13.2 Evaluation metrics

A. Scenario plausibility

How "realistic" are the generated conditional scenarios?

Metric	What it measures	How to compute
Mahalanobis distance	Are conditional samples within a plausible region of the data?	`d(x) = sqrt((x - μ)ᵀ Σ⁻¹ (x - μ))` using historical μ, Σ. Compare distribution of d for each generator
Energy distance	Two-sample test between conditional samples and historical conditional subset	`E[‖X-Y‖] - E[‖X-X'‖]/2 - E[‖Y-Y'‖]/2`. Lower = more similar to historical
Cross-factor consistency	Do non-conditioned factors behave realistically?	Compare conditional correlations from generated vs historical. Use Frobenius norm of correlation difference
Tail density ratio	Does the generator produce appropriate tail mass?	For each non-conditioned factor, compare P(factor < 1st percentile) between generated and historical

B. Scenario diversity and novelty

Can the generator create scenarios that didn't happen historically?

Metric	What it measures	How to compute
Coverage	Does the generator span the support of the historical distribution?	% of historical data points that have a generated neighbour within ε
Novelty	Does the generator create new plausible points?	% of generated samples that are NOT near any historical data point
Effective sample size	How many independent scenarios?	N_eff = N / (1 + 2Σ_k ρ(k)) or simpler: N / (1 + max pairwise correlation)

The diffusion model should beat historical rejection sampling on **novelty** while beating parametric models on **plausibility**.

C. Portfolio-relevant metrics

What a quant team actually cares about:

Metric	What it measures	How to compute
VaR accuracy	Is the conditional VaR estimate stable and accurate?	Generate M batches of conditional samples, compute VaR for each. Report mean and std of VaR across batches

Metric	What it measures	How to compute
ES accuracy	Same for expected shortfall	As above but for ES
Risk sensitivity	Does portfolio risk change in the right direction?	Compute ΔVaR when going from unconditional to conditional. Compare sign and magnitude across generators
Hedging utility	Can the scenarios identify useful hedges?	For a given portfolio, which stock/factor/group of stocks has the best risk reduction when added? Compare this across generators — do they agree?

D. Ablation: what does the diffusion model contribute?

Run this matrix:

Model	Noise	Residuals	Conditional
Historical bootstrap	—	—	Rejection sampling
Multivariate t	—	—	Analytic conditional
DDPM	Gaussian	Gaussian	Gradient guidance
DDPM	Gaussian	t-distributed	Gradient guidance
DLPM	Lévy (α=1.9)	Gaussian	Gradient guidance (new)
DLPM	Lévy (α=1.9)	t-distributed	Gradient guidance (new)

For each row: generate 4096 conditional samples for "momentum < -3%", compute all metrics above, compare.

This matrix answers:

1. **Row 3 vs 1**: Does DDPM beat rejection sampling? (should win on novelty and diversity)
2. **Row 3 vs 2**: Does DDPM beat parametric? (should win on plausibility and cross-factor consistency)
3. **Row 4 vs 3**: Do t-distributed residuals help? (should improve tail metrics at stock level)
4. **Row 5 vs 3**: Does Lévy noise help at factor level? (should improve factor-level tail metrics)
5. **Row 6 vs 4**: Does Lévy noise add value on top of t-residuals? (marginal improvement?)

13.4 Practical test recipe

```
Step 1: Pick a scenario → "momentum < -3%"
Step 2: Pick a portfolio → long momentum, equal-weight top 20 stocks by momentum beta
Step 3: For each generator:
    a. Generate 4096 conditional factor samples
    b. Reconstruct stock returns (betas + residuals)
    c. Compute portfolio return distribution
    d. Record: VaR 1%, ES 1%, mean, std, skew, kurtosis
    e. Record: cross-factor correlation matrix
    f. Record: Mahalanobis distances, energy distance to historical
Step 4: Make a comparison table + overlay histograms
Step 5: Repeat for 3-5 different scenarios to check robustness
Step 6: Repeat for S&P 500 dataset (more stocks, longer history)
```